

Inteligência Artificial

Licenciatura em Engenharia Informática

Relatório do Trabalho Prático

Jogos de Dois Jogadores



UNIVERSIDADE DE ÉVORA

Miguel Pombeiro, 57829 | Miguel Rocha, 58501

Departamento de Informática
Universidade de Évora
Maio 2026

Índice

1 Variante do jogo do Nim	2
1.1 Estrutura de dados	2
1.2 Predicado terminal	2
1.3 Função de utilidade	2
1.4 MiniMax	3
1.5 Alfa-Beta	4
1.6 Função de avaliação	4
1.7 Agente Inteligente	6
1.8 Nós Expandidos	7
2 Jogo dos Sapos e das Rãs	9
2.1 Estrutura de dados	9
2.2 Predicado terminal	9
2.3 Função de utilidade	10
2.4 Função de avaliação	10
2.5 Agente Inteligente	11
2.6 Nós expandidos	11
Instruções para Correr o Problema	12

1 Variante do jogo do Nim

O jogo do Nim é um jogo determinístico para dois jogadores. Nesta variante do jogo, existem duas pilhas de pedras, e para cada jogador, as regras são as seguintes:

- Retirar qualquer número de pedras de apenas uma das pilhas;
- Retirar o mesmo número de pedras de ambas as pilhas simultaneamente.
- Perde o jogador que já não tiver pedras para tirar.

1.1 Estrutura de dados

A estrutura de dados escolhida para o jogo consiste no número de pedras que estão em cada pilha e o turno do jogador.

Para o estado inicial indicado no enunciado, a pilha um contém 15 pedras, a pilha dois contém 10 pedras e o turno do jogador **a** ou **b**, conforme o jogador atual. Desta forma, em Prolog, o estado inicial é representado por:

```
estado_inicial(e(15, 10, a)).
```

1.2 Predicado terminal

Este jogo termina quando já não existirem mais pedras para retirar de nenhuma das pilhas. Em Prolog, o predicado `terminal/1` é o seguinte:

```
terminal(e(0, 0, _)).
```

1.3 Função de utilidade

A função de utilidade escolhida foi a seguinte:

$$\begin{cases} 50 - P, & \text{se o jogador inteligente ganha} \\ -50 + P, & \text{se o jogador inteligente perde} \end{cases}$$

*Considerando que **P** é a profundidade do estado terminal, de modo a dar prioridade a soluções que chegam a um estado terminal mais rápido. Assim, permite distinguir vitórias rápidas de vitórias lentas, ou derrotas rápidas de derrotas lentas.

Desta forma, considerando o jogador inteligente o jogador **a**, a função de utilidade pode ser representada em Prolog da seguinte forma:

```

valor(e(0, 0, b), V, P) :-
    V is 50 - P.
valor(e(0, 0, a), V, P) :-
    V is -50 + P.

```

1.4 MiniMax

1.4.1 Resultados

Para os cálculos de tempo e de espaço, foram utilizados os valores do número de nós visitados e do número máximo de nós em memória, respetivamente.

Tabela 1: Resultados obtidos para o Minimax.

Estado	Tempo	Espaço	Jogada Decidida
$e(15, 10, a)$	¹	¹	¹
$e(9, 14, a)$	¹	¹	¹
$e(12, 10, a)$	¹	¹	¹
$e(11, 8, a)$	¹	¹	¹
$e(8, 8, a)$	¹	¹	¹
$e(5, 10, a)$	6067815	16	retiraP2(7)
$e(6, 7, a)$	988297	14	retiraP12(5)
$e(5, 8, a)$	829464	14	retiraP2(5)
$e(7, 5, a)$	295548	13	retiraP1(4)
$e(6, 6, a)$	324556	13	retiraP12(6)
$e(7, 4, a)$	83187	12	retiraP2(1)
$e(4, 4, a)$	3606	9	retiraP12(4)
$e(3, 5, a)$	3152	9	retiraP2(4)
$e(2, 2, a)$	46	5	retiraP12(2)
$e(2, 1, a)$	15	4	retiraP2(1)
$e(1, 1, a)$	6	3	retiraP12(1)

¹ Não terminou em tempo útil (menos de cinco minutos).

1.5 Alfa-Beta

1.5.1 Resultados

Para os cálculos de tempo e de espaço, foram utilizados os valores do número de nós visitados e do número máximo de nós em memória, respetivamente.

Tabela 2: Resultados obtidos para o Alfa-Beta.

Estado	Tempo	Espaço	Jogada Decidida
$e(15, 10, a)$	$_{-1}^1$	$_{-1}^1$	$_{-1}^1$
$e(9, 14, a)$	$_{-1}^1$	$_{-1}^1$	$_{-1}^1$
$e(12, 10, a)$	518478	18	retiraP12(7)
$e(11, 8, a)$	165336	16	retiraP12(4)
$e(8, 8, a)$	17602	13	retiraP12(8)
$e(5, 10, a)$	1510657	16	retiraP2(7)
$e(6, 7, a)$	3341	11	retiraP12(5)
$e(5, 8, a)$	28218	13	retiraP2(5)
$e(7, 5, a)$	2803	11	retiraP12(2)
$e(6, 6, a)$	1904	10	retiraP12(6)
$e(7, 4, a)$	5312	11	retiraP12(1)
$e(4, 4, a)$	204	7	retiraP12(4)
$e(3, 5, a)$	839	9	retiraP12(1)
$e(2, 2, a)$	20	4	retiraP12(2)
$e(2, 1, a)$	11	4	retiraP12(1)
$e(1, 1, a)$	6	3	retiraP12(1)

¹ Ocorreu *Stack Overflow*, o que não permitiu terminar a execução.

Podemos concluir, assim, que em relação aos resultados da Tabela 1, o corte Alfa-Beta foi muito útil, permitindo não só obter solução para mais exemplos relativamente ao Minimax, como também reduzir significativamente o tempo de execução e o número máximo de nós em memória.

1.6 Função de avaliação

A função de avaliação utilizada para recorrer a algoritmos com corte em profundidade tem em conta se o jogador inteligente consegue ou não ganhar na jogada seguinte. Neste caso, é apenas testado se falta apenas um movimento para o agente inteligente ganhar ou perder. A função considerada foi a seguinte:

$$\text{Avaliação}(E) = \begin{cases} 1, & \text{se faltar um movimento para o jogador inteligente ganhar} \\ -1, & \text{se faltar um movimento para o jogador inteligente perder} \\ 0, & \text{caso contrário, indeterminado} \end{cases}$$

Em Prolog:

```

avalia(e(X, X, a), 1).
avalia(e(0, _, a), 1).
avalia(e(_, 0, a), 1).

avalia(e(X, X, b), -1).
avalia(e(0, _, b), -1).
avalia(e(_, 0, b), -1).

avalia(e(X, Y, _), 0):-
    X > 0,
    Y > 0,
    X \= Y.

```

1.6.1 Resultados

Para os cálculos de tempo e de espaço, foram utilizados os valores do número de nós visitados e do número máximo de nós em memória, respetivamente.

Tabela 3: Resultados obtidos para o Minimax com corte à profundidade 6.

Estado	Tempo	Espaço	Jogada Decidida
$e(15, 10, a)$	10371008 ¹	7 ¹	retiraP2(6) ¹
$e(9, 14, a)$	5847293	7	retiraP2(10)
$e(12, 10, a)$	4700901	7	retiraP12(7)
$e(11, 8, a)$	1671894	7	retiraP12(4)
$e(8, 8, a)$	532111	7	retiraP12(8)
$e(5, 10, a)$	264925	7	retiraP2(7)
$e(6, 7, a)$	118581	7	retiraP12(5)
$e(5, 8, a)$	107208	7	retiraP2(5)
$e(7, 5, a)$	62920	7	retiraP1(4)
$e(6, 6, a)$	66718	7	retiraP12(6)
$e(7, 4, a)$	29803	7	retiraP2(1)
$e(4, 4, a)$	3116	7	retiraP12(4)
$e(3, 5, a)$	2753	7	retiraP2(4)
$e(2, 2, a)$	46	5	retiraP12(2)
$e(2, 1, a)$	15	4	retiraP2(1)
$e(1, 1, a)$	6	3	retiraP12(1)

¹ Demorou cerca de 7m42s para executar.

Comparando estes resultados em relação à versão sem cortes em profundidade, representada na Tabela 1, foi possível chegar a uma solução para mais quatro exemplos. Para além disso, o número de nós visitados diminuiu significativamente (e.g. no estado $e(5, 10, a)$ de 6067815 para 264925).

Tabela 4: Resultados obtidos para o Alfa-Beta com corte à profundidade 6.

Estado	Tempo	Espaço	Jogada Decidida
$e(15, 10, a)$	167854	7	retiraP12(1)
$e(9, 14, a)$	204327	7	retiraP12(1)
$e(12, 10, a)$	23214	7	retiraP12(7)
$e(11, 8, a)$	27497	7	retiraP12(4)
$e(8, 8, a)$	4757	7	retiraP12(8)
$e(5, 10, a)$	55315	7	retiraP2(7)
$e(6, 7, a)$	1951	7	retiraP12(5)
$e(5, 8, a)$	9956	7	retiraP2(5)
$e(7, 5, a)$	2039	7	retiraP12(2)
$e(6, 6, a)$	1340	7	retiraP12(6)
$e(7, 4, a)$	3627	7	retiraP12(1)
$e(4, 4, a)$	204	7	retiraP12(4)
$e(3, 5, a)$	833	7	retiraP12(1)
$e(2, 2, a)$	20	4	retiraP12(2)
$e(2, 1, a)$	11	4	retiraP12(1)
$e(1, 1, a)$	6	3	retiraP12(1)

Comparando estes resultados em relação à versão AlfaBeta sem cortes em profundidade, representada na Tabela 2, o número de nós visitados diminuiu significativamente (e.g. no estado $e(5, 10, a)$ de 1510657 para 55315). O tempo de execução também seguiu a mesma tendência.

No geral, a utilização de cortes em profundidade acabou por diminuir bastante o tempo de execução para encontrar uma solução, independentemente da dimensão do estado inicial. Diminuir para uma profundidade ainda menor (e.g. 5) permitiria obter resultados ainda mais rapidamente, mas poderia piorar a qualidade da solução obtida.

1.7 Agente Inteligente

O agente inteligente deste problema está implementado no ficheiro `agente.pl`. Este agente implementa o ciclo de jogo, em que o agente inteligente começa a jogar primeiro e depois pede uma jogada ao utilizador. O jogo continua até ser atingido um estado terminal, sendo, no final, exibido qual dos jogadores ganhou.

Existem dois agentes possíveis (`agente.pl` e `agenteJog.pl`) e, apesar de qualquer um conseguir jogar ambos os problemas, é recomendado utilizar `agente.pl` para esta variante do problema do Nim. O outro ficheiro é descrito em mais detalhe na Secção 2.5.

Para executar o agente deve seguir as seguintes instruções:

1. **Consultar o ficheiro do agente:** `[agenteJog].` ou `[agente].`
2. **Escolher o problema e a estratégia:** Deve-se usar os nomes dos respetivos ficheiros.
3. **Correr o agente:** ex.: `agente(problema1, minmax).`
4. **Escolher a jogada:** ex.: `retiraP1(3).`

De notar que estão implementados quatro algoritmos diferentes, cada um num ficheiro Prolog diferente:

- **Minimax:** `minmax.pl`
- **Minimax com corte em profundidade:** `minmaxCorte.pl`
- **Minimax com corte alfa-beta:** `mmAlfaBeta.pl`
- **Minimax com corte alfa-beta e em profundidade:** `mmCAlfaBeta.pl`

De acordo com os resultados obtidos na Subsecção 1.6, é recomendado usar o Alfa-Beta com cortes de modo a melhorar a experiência de utilização, ou então diminuir a profundidade em que se está a realizar o corte.

1.8 Nós Expandidos

Para o cálculo dos estados diferentes que estão nos nós da árvore minimax, foi utilizada uma fórmula que conta todos os estados diferentes teóricos totais e que retira o número de estados inalcançáveis pelo jogador que não começa. Neste caso, o número de estados teóricos totais são as combinações de pedras de ambas as pilhas, multiplicadas pelo número de jogadores (2). Os estados inalcançáveis pelo jogador não inicial são o estado inicial do problema, e os casos em que é retirada uma única peça das pilhas (se for possível).

A fórmula usada foi a seguinte:

$$N(\text{estado_inicial}(X, Y, _)) = \begin{cases} 2 \cdot (X + 1) \cdot (Y + 1) - 3, & \text{se } X, Y \neq 0 \\ 2 \cdot (X + 1) - 2, & \text{se } X \neq 0, Y = 0 \\ 2 \cdot (Y + 1) - 2, & \text{se } X = 0, Y \neq 0 \\ 1, & \text{se } X, Y = 0 \end{cases}$$

Os valores foram, posteriormente confirmados de através da execução do programa.

Tabela 5: Nós expandidos para encontrar a solução com os vários algoritmos. Para os algoritmos com corte, este foi feito à profundidade 6.

Estado	MinMax	MinMax Corte	AlfaBeta	AlfaBeta Corte	E. Diferentes
$e(15, 10, a)$	- ¹	1346863	- ²	51974	349 ³
$e(9, 14, a)$	- ¹	840085	- ²	56339	280 ³
$e(12, 10, a)$	- ¹	702949	324128	9727	283 ³
$e(11, 8, a)$	- ¹	299742	99774	11980	213 ³
$e(8, 8, a)$	- ¹	116616	11072	2316	159 ³
$e(5, 10, a)$	3170748	65115	791109	17288	129
$e(6, 7, a)$	518064	33536	2096	1050	109
$e(5, 8, a)$	434298	30750	15619	4536	105
$e(7, 5, a)$	154890	19711	1685	1082	93
$e(6, 6, a)$	170204	20736	1192	744	95
$e(7, 4, a)$	43562	10463	2978	1716	77
$e(4, 4, a)$	1894	1474	126	126	47
$e(3, 5, a)$	1654	1311	479	426	45
$e(2, 2, a)$	24	24	12	12	15
$e(2, 1, a)$	8	8	6	6	9
$e(1, 1, a)$	3	3	3	3	5

¹ Não terminou em tempo útil (menos de cinco minutos).

² Ocorreu *Stack Overflow*, o que não permitiu terminar a execução.

³ Não foi possível confirmar estes valores computacionalmente uma vez que a execução não terminou em tempo útil.

2 Jogo dos Sapos e das Rãs

O jogo de dois jogadores determinístico e com informação completa escolhido foi o jogo dos Sapos e das Rãs (*Toads and Frogs*).

Este jogo pode ser jogado num tabuleiro $1 \times n$, e a cada momento, cada quadrado deste tabuleiro pode estar vazio ou ocupado por um único sapo, ou rã.

A configuração escolhida para este jogo foi um tabuleiro 1×9 , e a representação dos sapos do lado esquerdo e as rãs do lado direito.

Considerando a vez do jogador da esquerda, as regras são as seguintes:

- Pode mover um sapo um quadrado para a direita, para um quadrado vazio.
- Pode "saltar" um sapo dois quadrados para a direita, sobre uma rã, para um quadrado vazio.
- Saltos sobre um quadrado vazio, um sapo ou mais do que um quadrado não são permitidos.
- Caso não existam movimentos possíveis, perde.

Para o jogador da direita, as regras são análogas, pode mover uma rã para a esquerda, para um espaço vazio adjacente, ou fazer saltar uma rã sobre um único sapo para o quadrado vazio imediatamente à esquerda do sapo.

Estas regras foram retiradas da Wikipedia.

2.1 Estrutura de dados

A estrutura de dados escolhida para o jogo é composta pelo tabuleiro do jogo e o turno do jogador.

Este tabuleiro é composto por 3 sapos (*s*), 3 espaços vazios (*v*) e 3 rãs (*r*), e o turno do jogador *s* ou *v*, conforme seja o sapo a jogar ou a rã a jogar. O estado inicial representado em Prolog é o seguinte:

```
estado_inicial(e([s,s,s,v,v,v,r,r,r], s)).
```

2.2 Predicado terminal

Este jogo termina quando o jogador não tem mais jogadas possíveis, e o predicado `terminal/1` pode ser representado da seguinte maneira em Prolog:

```
terminal(E) :-  
    \+ op1(E, _, _).
```

2.3 Função de utilidade

A função de utilidade escolhida foi a seguinte:

$$\text{Utilidade}(E) = \begin{cases} 50 - P, & \text{se o jogador inteligente ganha} \\ -50 + P, & \text{se o jogador inteligente perde} \end{cases}$$

*Considerando que P é a profundidade do estado terminal, de modo a dar prioridade a soluções que chegam a um estado terminal mais rápido. Assim, permite distinguir vitórias rápidas de vitórias lentas, ou derrotas rápidas de derrotas lentas.

Considerando o jogador inteligente o sapo, (s), a função de utilidade pode ser representada em Prolog na seguinte maneira:

```
valor(e(_, r), V, P):-  
    V is 50 - P.
```

```
valor(e(_, s), V, P):-  
    V is -50 + P.
```

2.4 Função de avaliação

De modo a poder testar este problema em algoritmos com corte em profundidade, foi necessário implementar a seguinte função de avaliação:

$$\text{Avaliação}(E) = |J_{\text{sapo}}(E)| - |J_{\text{rã}}(E)|$$

onde $J_{\text{sapo}}(E)$ representa o conjunto de jogadas possíveis do sapo no estado E , e $J_{\text{rã}}(E)$ representa o conjunto de jogadas possíveis da rã.

Em Prolog:

```
avalia(e(T, _), Valor) :-  
    mov_restantes(T, s, MovS),  
    mov_restantes(T, r, MovR),  
    Valor is MovS - MovR.  
  
mov_restantes(T, J, NMov) :-  
    findall(Mov, op1(e(T, J), Mov, _), ListaMovimentos),  
    length(ListaMovimentos, NMov).
```

2.5 Agente Inteligente

O agente inteligente deste problema está implementado no ficheiro `agenteJog.pl`, que ao contrário do `agente.pl`, imprime as jogadas possíveis de modo a auxiliar o jogador.

Apesar de haver dois agentes possíveis, qualquer um consegue jogar ambos os problemas, sendo recomendado usar o `agenteJog.pl` porque este problema apresenta muito menos jogadas possíveis em cada turno.

O agente tem as seguintes instruções:

1. **Consultar o ficheiro do agente:** `[agenteJog].` ou `[agente].`
2. **Escolher o problema e a estratégia:** Deve-se usar os nomes dos respetivos ficheiros.
3. **Correr o agente:** ex.: `agente(problema2, minmax).`
4. **Escolher a jogada:** ex.: `anda_r(7,6).`

2.6 Nós expandidos

Para simplificar a tabela dos nós expandidos (Tabela 7), foi realizada uma tabela auxiliar (Tabela 6), que faz a correspondência dos estados a um identificador.

Tabela 6: Correspondência entre os identificadores dos estados e os estados utilizados nos testes.

Identificador	Estado
E_1	<code>e([s,s,s,v,v,v,r,r,r],s)</code>
E_2	<code>e([s,s,v,s,v,r,v,r,r],s)</code>
E_3	<code>e([s,v,s,s,r,v,v,r,r],s)</code>
E_4	<code>e([s,v,s,s,v,r,r,v,r],s)</code>
E_5	<code>e([s,s,v,r,s,v,v,r,r],s)</code>
E_6	<code>e([s,s,v,v,s,r,r,v,r],s)</code>
E_7	<code>e([v,s,s,s,r,v,r,v,r],s)</code>
E_8	<code>e([v,s,s,s,v,r,r,r,v],s)</code>
E_9	<code>e([s,v,s,r,s,v,r,v,r],s)</code>
E_{10}	<code>e([s,v,s,v,s,r,r,r,v],s)</code>
E_{11}	<code>e([s,r,s,v,s,v,v,r,r],s)</code>
E_{12}	<code>e([s,s,r,v,v,s,v,r,r],s)</code>
E_{13}	<code>e([s,s,v,r,v,s,r,v,r],s)</code>
E_{14}	<code>e([s,v,s,r,s,v,r,u,r],s)</code>

Tabela 7: Nós expandidos para encontrar a solução com os vários algoritmos. Para os algoritmos com corte foi considerada a profundidade 6.

Estado	MinMax	MinMax Corte	AlfaBeta	AlfaBeta Corte	E. Diferentes
E_1	70184	16	1864	13	819
E_2	70182	50	1862	25	817
E_3	26007	50	1292	42	640
E_4	2698	34	475	28	383
E_5	38989	58	1818	35	690
E_6	2485	22	388	16	320
E_7	205	16	94	16	128
E_8	7	6	7	6	8
E_9	2476	42	379	29	315
E_{10}	7	6	7	6	9
E_{11}	4180	27	611	23	344
E_{12}	25186	63	1456	50	577
E_{13}	7144	49	476	24	392
E_{14}	17	14	13	11	19

Instruções para Correr o Problema

Para além das instruções fornecidas para correr os agentes, `agente.pl` e `agenteJog.pl`, já referidos na Subseção 1.7 e Subseção 2.5, é também necessário saber as instruções para correr os ficheiros manualmente.

Nos ficheiros dos problemas (`problema1.pl` ou `problema2.pl`) basta apenas "descomentar" o estado inicial que se quer utilizar.

Para correr os problemas, é apenas necessário consultar o ficheiro do algoritmo, cujos nomes estão referidos na Subseção 1.7 e correr `g(problema1)` ou `g(problema2)`.